

GUIDE DU STAGIAIRE

DOCKER GRÉER LE NAVIRE

Registry, volumes, réseaux et Compose. Vos consignes, concepts, commandes à exécuter, quiz et espaces de notes. Les corrigés sont présentés en séance.

6

MISSIONS

1

STACK COMPOSE

2

RÉSEAUX



SPARKS
FORMATION

Guide stagiaire · Jour 2
Session inter · du 06/07/2026 au 08/07/2026
Docker Engine 29.x · Compose v2 · Swarm

Gréer le navire

Registry, volumes, réseaux et Compose

Au programme aujourd'hui

Aujourd'hui, ShipFast gagne un superpouvoir à chaque mission : vous allez apprendre à la partager (registry privé), à lui donner de la mémoire (volumes), à la câbler sur un réseau sécurisé, puis à assembler tout cela d'un seul geste avec Docker Compose. Vous terminerez la journée avec une stack complète, capable de grandir comme une vraie équipe de développement le ferait.

À l'issue de cette journée, vous serez capable de :

- publier et récupérer une image depuis votre propre registry privé ;
- choisir entre volume nommé, bind mount et tmpfs selon le besoin, et sauvegarder/restaurer des données ;
- construire une architecture réseau front/back isolée, avec résolution DNS interne ;
- écrire un fichier `compose.yaml` complet, avec healthchecks et dépendances ;
- diagnostiquer une stack Compose en panne ;
- faire évoluer une stack avec profiles, watch, variables d'environnement et scaling.

HORAIRE	ACTIVITÉ	OBJECTIF
9h00 - 9h15	Quiz réveil + plan de journée	—
9h15 - 9h45	Exercice 1 — La boîte à outils : 4 défis express	4 outils du quotidien, sans rien installer
9h45 - 10h35	Exercice 2 — Votre propre registry privé	Un registry d'entreprise + ShipFast publié
10h35 - 10h50	Pause	—
10h50 - 12h10	Exercice 3 — Volumes : greffer une mémoire à ShipFast (+ crash test)	Une base PostgreSQL qui survit à tout, sauvegardée
12h10 - 12h30	Débrief volumes : bind / volume / tmpfs, sauvegardes, cycle de vie	—
13h30 - 13h40	Échauffement « 3 commandes, 3 réseaux »	—
13h40 - 14h50	Exercice 4 — Réseaux : câbler ShipFast à la main	L'architecture front/back complète, sécurisée
14h50 - 15h05	Pause	—
15h05 - 16h05	Exercice 5 — Compose : tout ShipFast en une commande + « Casse-prod »	Le compose.yaml de référence du fil rouge
16h05 - 16h40	Exercice 6 — Faire grandir la stack : Adminer, watch, variables, scale	Une stack de dev complète et confortable

HORAIRE	ACTIVITÉ	OBJECTIF
16h40 - 17h00	Quiz de clôture J2 + teaser J3	—

Les concepts du jour

Les volumes : donner une mémoire à un conteneur

Par défaut, tout ce qu'un conteneur écrit disparaît quand vous le détruisez : la couche d'écriture est jetable. Trois mécanismes permettent de faire persister des données au-delà de la vie d'un conteneur :

- **Volume nommé** : un espace de stockage géré entièrement par Docker (`docker volume create`), monté dans le conteneur. C'est le mécanisme recommandé pour les données d'un conteneur (bases de données, registry, etc.) : il survit à la destruction du conteneur, peut être sauvegardé, et Docker le pré-remplit avec le contenu de l'image au premier montage.
- **Bind mount** : un dossier de votre machine hôte est monté directement dans le conteneur (`-v "$PWD":/app`). C'est l'outil du développeur : idéal pour le code source que vous modifiez dans votre éditeur, ou un fichier de configuration.
- **tmpfs** : un montage en RAM pure (Linux), pour des fichiers temporaires ou sensibles qui ne doivent jamais toucher le disque.

À RETENIR

Règle mémorisable : « les volumes pour les données du conteneur, les bind mounts pour les fichiers de l'humain ». Un volume survit à ses conteneurs (`docker volume ls` , `docker volume prune` — à manier avec précaution, cela supprime des données). Deux méthodes de sauvegarde : la sauvegarde « à froid » d'un volume (archiver son contenu avec un conteneur temporaire), ou la sauvegarde logique via l'outil de la base de données (par exemple `pg_dump`) — la bonne méthode pour une base vivante.

L'isolation réseau front/back

Docker fournit plusieurs types de réseaux : `bridge` (le mode par défaut, NAT privé), `host` (aucune isolation, performance maximale, Linux uniquement), `none` (aucune interface réseau, isolation totale), `overlay` (multi-machines, vu demain avec Swarm) et `macvlan` (le conteneur reçoit une vraie IP du réseau local).

Sur un réseau que vous créez vous-même (`docker network create`), Docker active une résolution DNS interne : les conteneurs peuvent se joindre par leur nom (par exemple `db`), sans connaître d'adresse IP. Ce n'est pas le cas sur le réseau `bridge` par défaut — d'où la règle : toujours créer son propre réseau. Un réseau créé avec l'option `--internal` n'a aucun accès à Internet : parfait pour isoler une base de données ou un cache qui n'ont besoin de parler qu'aux autres conteneurs.

À RETENIR

Une architecture sécurisée sépare un réseau `front` (le proxy, seul point d'entrée public) d'un réseau `back`, marqué `--internal`, où vivent la base de données et le cache. Seul le composant qui a besoin des deux (l'API) est branché sur les deux réseaux. La base de données ne doit jamais publier de port sur l'hôte : rien ne justifie qu'elle soit joignable depuis l'extérieur.

Le déclic Compose : un fichier, une commande

Construire une architecture multi-conteneurs « à la main » (comme dans l'exercice 4) exige une longue suite de commandes `docker run`, `docker network create`, `docker network connect`... difficile à relancer, à partager avec un collègue, ou à versionner. Docker Compose résout exactement ce problème : un seul fichier déclaratif (`compose.yaml`) décrit tous les services, réseaux et volumes de votre application, et une seule commande (`docker compose up`) suffit à tout démarrer.

À RETENIR

Compose ne remplace pas les concepts vus dans la journée (volumes, réseaux, healthchecks) : il les déclare dans un fichier au lieu de les taper à la main. `depends_on` combiné à `condition: service_healthy` garantit qu'un service ne démarre que lorsque sa dépendance est réellement prête (pas seulement lancée). `docker compose down` ne supprime pas les volumes, sauf si vous ajoutez explicitement `-v`.

NOTES

Exercices

EXERCICE 1 · 9H15-9H45 · 30 MIN

La boîte à outils : 4 défis express

Objectif : découvrir le réflexe « pourquoi installer, quand un conteneur jetable fait le job ? » à travers quatre défis rapides.

CE QUE VOUS DEVEZ FAIRE

Réalisez les 4 défis ci-dessous, environ 6 minutes chacun. Contrainte absolue : interdiction d'installer quoi que ce soit sur votre machine — utilisez uniquement des conteneurs jetables (`--rm`). Pour chaque défi, notez la commande utilisée et la preuve que ça fonctionne.

```
# DÉFI 1 – "Partage éclair" : servez le dossier moncv/ d'hier sur le port 8000,  
# sans build, en UNE commande. (Indice : hier on a construit ;  
# aujourd'hui on peut simplement... monter.)  
  
# DÉFI 2 – "Le stagiaire a besoin de Python" : calculez 2**100 avec Python 3.14  
# alors que Python n'est (peut-être) pas installé sur votre poste.
```

```
# DÉFI 3 – "Le rapport de la direction" : convertissez le README.md du dossier
#      shipfast en HTML avec pandoc – que personne n'a envie d'installer.

# DÉFI 4 – "Le vieux projet" : votre client tourne encore sur Node 16 (2021).
#      Prouvez en une commande que le server.js de ShipFast démarre
#      (ou pas !) sous Node 16, sans toucher à votre Node local.
```

Questions à préparer :

- Quel est le pattern commun aux 4 défis (les options utilisées à chaque fois) ?
- Pour le défi 4 : dans quel contexte professionnel ce type de test est-il particulièrement utile ?

NOTES

EXERCICE 2 · 9H45-10H35 · 50 MIN

Votre propre registry privé

Objectif : monter un registry d'images privé et y publier ShipFast, en comprenant la mécanique registre/tag/push/pull.

CE QUE VOUS DEVEZ FAIRE

Votre entreprise interdit de pousser les images sur Docker Hub (propriété intellectuelle et limites de pull). Montez un registry privé et publiez-y ShipFast. Suivez les étapes ci-dessous, puis répondez à la question d'observation.

```
# 1. Démarrer le registry (image officielle du projet CNCF "distribution")
docker run -d --name registry -p 5000:5000 \
  -v registry-data:/var/lib/registry registry:3

# 2. Publier : le nom de l'image EST son adresse
docker tag shipfast:v2 localhost:5000/shipfast:v2
docker push localhost:5000/shipfast:v2

# 3. Vérifier via l'API HTTP du registry
curl http://localhost:5000/v2/_catalog
curl http://localhost:5000/v2/shipfast/tags/list

# 4. La preuve par le feu : supprimer l'image locale et la récupérer
docker rmi localhost:5000/shipfast:v2 shipfast:v2
docker pull localhost:5000/shipfast:v2
docker run -d --rm -p 3000:3000 localhost:5000/shipfast:v2 && curl localhost:3000/stats

# 5. Publiez aussi votre carte de visite d'hier (moncv:v2) et vérifiez
#    le catalogue : votre registry a maintenant 2 dépôts.

# 6. Question d'observation (bonus) : redémarrez le registry
#    (docker rm -f registry, puis relancez la commande 1).
#    Vos images sont-elles encore là ? Pourquoi ?
```

Questions à préparer :

- Que signifie exactement un nom complet d'image du type `registre[:port]/dépôt/nom:tag` ? Que représente `nginx` en réalité ?
- Quelles limites présente ce registry « nu » par rapport à une solution d'entreprise (Harbor, ECR, GitLab Registry) ?

DÉFI BONUS

Redémarrez le registry sans le volume `-v registry-data:/var/lib/registry` et observez ce qu'il advient de vos images. Expliquez pourquoi.

NOTES

EXERCICE 3 · 10H50-12H10 · 80 MIN

Volumes : greffer une mémoire à ShipFast

Objectif : construire une base PostgreSQL qui survit à la destruction de son conteneur, tester une procédure de sauvegarde/restauration, et monter un environnement de développement à hot-reload.

Acte 1 — Constater le problème

CE QUE VOUS DEVEZ FAIRE

Exécutez la séquence ci-dessous et observez ce qui se passe quand vous détruisez et recréez le conteneur de base de données sans volume.

```
docker run -d --name db -e POSTGRES_PASSWORD=formation postgres:18-alpine
docker exec -it db psql -U postgres -c "CREATE TABLE tresor(id int); INSERT INTO tresor
VALUES (1);"
docker rm -f db
docker run -d --name db -e POSTGRES_PASSWORD=formation postgres:18-alpine
docker exec -it db psql -U postgres -c "SELECT * FROM tresor;"
# Q1 : que constatez-vous ? Pourquoi ?
```

Acte 2 — La persistance

CE QUE VOUS DEVEZ FAIRE

Recommencez avec un volume nommé, vérifiez que les données survivent à un crash test (destruction du conteneur), puis pratiquez les deux méthodes de sauvegarde.

```
# 1. Un volume nommé, géré par Docker
docker volume create pgdata
docker volume inspect pgdata          # → Mountpoint : où vit-il réellement ?

# 2. PostgreSQL avec mémoire (le -v change tout)
docker rm -f db
docker run -d --name db -e POSTGRES_PASSWORD=formation \
-v pgdata:/var/lib/postgresql/data postgres:18-alpine
```

```
# 3. Créez la table "tresor" comme dans l'acte 1, puis... CRASH TEST :
docker rm -f db          # on détruit tout
docker run -d --name db -e POSTGRES_PASSWORD=formation \
  -v pgdata:/var/lib/postgresql/data postgres:18-alpine
docker exec -it db psql -U postgres -c "SELECT * FROM tresor;"

# 4. Sauvegarde à froid d'un volume (le pattern universel, à connaître par cœur)
docker run --rm -v pgdata:/data -v "$PWD":/backup alpine \
  tar czf /backup/pgdata-$(date +%F).tar.gz -C /data .
ls -lh pgdata-*.tar.gz

# 5. Sauvegarde logique (la bonne méthode pour une BDD vivante)
docker exec db pg_dump -U postgres postgres > dump.sql

# 6. BONUS : détruisez conteneur ET volume (docker volume rm pgdata),
# puis restaurez la table tresor uniquement à partir de dump.sql.
```

Acte 3 — Le bind mount, outil du développeur

CE QUE VOUS DEVEZ FAIRE

Lancez votre API avec le code source monté depuis votre machine, modifiez `server.js` dans votre éditeur (changez le message de la route `/stats`), et constatez le changement sans rebuild.

```
docker run -d --name ship-dev -p 3000:3000 \
  -v "$PWD":/app -w /app node:24-alpine \
  sh -c "npm ci && node --watch server.js"
# Modifier server.js → node --watch redémarre → curl localhost:3000/stats
```

Questions à préparer :

- Quelle est la différence d'usage entre un volume nommé et un bind mount ?
- Pourquoi la sauvegarde logique (`pg_dump`) est-elle préférable à la sauvegarde à froid pour une base de données vivante ?
- Qu'est-ce qui remplit automatiquement un volume nommé vide au premier montage ?

DÉFI BONUS

Détruisez à la fois le conteneur et le volume (`docker volume rm pgdata`), puis restaurez la table `tresor` uniquement à partir de votre fichier `dump.sql` .

NOTES

Câbler ShipFast à la main (et comprendre pourquoi Compose existe)

Objectif : construire l'architecture cible complète de ShipFast — proxy, API, base de données et cache — avec isolation front/back et DNS interne.

CE QUE VOUS DEVEZ FAIRE

Architecture cible : nginx (seul point d'entrée, port 8080) → API → PostgreSQL + Redis.
 Contrainte de sécurité : la base et le cache doivent être inaccessibles depuis le réseau du proxy. Vous avez le droit à deux réseaux : front et back. Le fichier `nginx.conf` est fourni dans le dossier du fil rouge.

```
# 1. Les deux réseaux (le réseau "back" est même privé d'Internet : --internal)
docker network create front
docker network create --internal back

# 2. Les données (réseau back uniquement)
docker run -d --name db --network back -e POSTGRES_PASSWORD=formation \
  -e POSTGRES_DB=shipfast -v pgdata2:/var/lib/postgresql/data postgres:18-alpine
docker run -d --name cache --network back redis:8-alpine

# 3. L'API : un pied dans chaque réseau
docker run -d --name api --network back \
  -e DATABASE_URL=postgres://postgres:formation@db:5432/shipfast \
  -e REDIS_URL=redis://cache:6379 \
  localhost:5000/shipfast:v2
docker network connect front api

# 4. Le proxy : réseau front, seul à publier un port
docker run -d --name proxy --network front -p 8080:80 \
  -v "$PWD/nginx.conf":/etc/nginx/conf.d/default.conf:ro nginx:1.30-alpine

# 5. Validation fonctionnelle
curl -X POST localhost:8080/links -H "Content-Type: application/json" \
  -d '{"url":"https://docs.docker.com"}'
curl localhost:8080/stats          # → "store":"postgres" : la persistance est branchée !

# 6. Validation SÉCURITÉ (la partie la plus importante)
docker exec proxy ping -c1 db      # → doit ÉCHOUER (réseaux différents)
docker exec api ping -c1 db        # → OK
docker exec db ping -c1 1.1.1.1    # → doit ÉCHOUER (réseau --internal)
```

ATTENTION

Il est possible que l'API démarre avant que PostgreSQL soit prêt à accepter des connexions, provoquant une erreur au premier appel (`docker logs api` montrera quelque chose comme « ECONNREFUSED »). Notez ce comportement si vous le rencontrez : vous le résoudrez proprement à l'exercice 5 avec `depends_on` et un healthcheck.

Questions à préparer :

- Comment l'API a-t-elle trouvé « db » sans connaître son adresse IP ?
- Pourquoi ne publie-t-on pas le port 5432 sur l'hôte ?
- Combien de commandes avez-vous dû exécuter pour monter cette stack ? Combien faudrait-il en retaper pour la reproduire, la partager avec un collègue, ou la versionner ?

NOTES

EXERCICE 5 · 15H05-16H05 · 60 MIN

Compose : tout ShipFast en une commande

Objectif : écrire le fichier `compose.yaml` de référence du fil rouge, puis déboguer une stack Compose sabotée.

Phase 1 — Écriture guidée (35 min)

CE QUE VOUS DEVEZ FAIRE

Faites un nettoyage général (`docker rm -f api db cache proxy`, en gardant les volumes). Traduisez toute l'architecture de l'exercice 4 en un fichier `compose.yaml`. Partez d'une page blanche — appuyez-vous sur la documentation officielle de Compose. Critère de réussite : `docker compose up -d` puis `curl localhost:8080/stats` répond, l'interface web s'affiche dans le navigateur, et les 3 tests de sécurité de l'exercice 4 passent toujours. Pensez à résoudre le problème de démarrage prématuré de l'API à l'aide de `depends_on` et d'un `healthcheck` — cherchez du côté de `condition: service_healthy`.

```
# Rappel des critères de validation
docker compose up -d
curl localhost:8080/stats
# Q : votre compose.yaml passe-t-il les 3 tests de sécurité de l'exercice 4 ?
```

Phase 2 — Défi « Casse-prod » (20 min)

CE QUE VOUS DEVEZ FAIRE

Vous allez recevoir un `compose.yaml` saboté de 5 façons différentes. Récupérez-le et faites fonctionner de nouveau `curl localhost:8080/stats`. Vous devrez lister les 5 sabotages que vous avez trouvés et corrigés. Utilisez la méthode : `docker compose config` pour valider le fichier, `docker compose ps` pour l'état des services, `docker compose logs -f [service]` pour les journaux, `docker compose exec service sh` pour investiguer à l'intérieur d'un service.

```
# Vos outils de diagnostic pour "Casse-prod"
docker compose config
docker compose ps
docker compose logs -f [service]
```

```
docker compose exec [service] sh
# Listez ici les 5 sabotages trouvés et la correction appliquée à chacun.
```

Questions à préparer :

- Pour chaque sabotage trouvé, quel symptôme vous a mis sur la piste (erreur de syntaxe, erreur réseau, erreur applicative...) ?
- Quel est l'ordre de réflexes de diagnostic le plus efficace face à une stack Compose qui ne fonctionne pas ?

NOTES

EXERCICE 6 · 16H05-16H40 · 35 MIN

Faire grandir la stack : Adminer, watch, variables, scale

Objectif : ajouter à votre stack un outil d'administration à la demande, du hot-reload officiel, des variables d'environnement propres, et un premier scaling.

Évolution 1 — L'outil d'admin sous profile

CE QUE VOUS DEVEZ FAIRE

Ajoutez le service `adminer` ci-dessous à votre `compose.yaml`, sous un profile `debug` pour qu'il ne se lance pas par défaut. Lancez votre stack normalement, puis avec le profile activé, et connectez-vous à Adminer pour consulter vos données.

```
# À ajouter dans compose.yaml :
adminer:
  image: adminer:latest
  ports: ["8082:8080"]
  networks: [back, front]      # back pour joindre db, front pour être joint
  profiles: ["debug"]          # ← n'existe QUE quand on le demande

docker compose up -d           # adminer absent : le quotidien reste léger
docker compose --profile debug up -d  # adminer présent
# → http://localhost:8082 : système PostgreSQL, serveur "db", postgres/formation
```

Évolution 2 — Le hot-reload officiel : compose watch

CE QUE VOUS DEVEZ FAIRE

Ajoutez la section `develop.watch` ci-dessous au service `api`, lancez `docker compose watch`, puis modifiez `server.js` et observez le comportement.

```
# À ajouter au service api :
develop:
  watch:
    - action: sync+restart
      path: ./server.js
      target: /app/server.js
```

```
docker compose watch      # modifiez server.js → sync + restart automatiques
```

Évolution 3 — Les variables : un compose qui s'adapte

CE QUE VOUS DEVEZ FAIRE

Créez un fichier `.env` à côté de votre `compose.yaml`, référencez ses variables dans le fichier, puis vérifiez la substitution et testez un changement de port sans modifier le fichier.

```
# .env (à créer à côté du compose.yaml)
TAG=v3
HTTP_PORT=8080

# Dans compose.yaml : image: localhost:5000/shipfast:${TAG:-v3}
#                      ports: [ "${HTTP_PORT:-8080}:80" ]
docker compose config | grep -E "image:|8080"    # → substitution visible
HTTP_PORT=9090 docker compose up -d            # → même fichier, autre port
```

Évolution 4 — Premier scaling

CE QUE VOUS DEVEZ FAIRE

Lancez votre API en 3 réplicas et observez la répartition de charge via le DNS interne.

```
docker compose up -d --scale api=3
docker compose ps                # 3 conteneurs api – possible car api ne publie pas de
port
for i in 1 2 3 4 5 6; do curl -s localhost:8080/stats | grep -o '"host":"[^"]*"'; done
# Q : combien de hostnames différents observez-vous ? Que cela prouve-t-il ?
```

Questions à préparer :

- Pourquoi `adminer` n'apparaît-il pas au lancement normal de la stack ?
- En quoi `compose watch` est-il une version « industrialisée » du `bind mount` utilisé à l'exercice 3 ?
- Quelles sont les limites honnêtes du scaling local vu ici (mono-machine, pas d'auto-réparation, pas de mise à jour sans coupure) ?

DÉFI BONUS

Installez Portainer (cockpit web de votre Docker) et explorez votre stack ShipFast entièrement depuis l'interface graphique : conteneurs, réseaux, volumes, et redéploiement du compose depuis le navigateur.

```
docker volume create portainer_data
docker run -d --name portainer --restart unless-stopped -p 9443:9443 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data portainer/portainer-ce:latest
# → https://vmX...:9443 (certificat auto-signé : à accepter) → créer le compte admin
```

Attention : créez le compte administrateur dans les 5 minutes suivant le démarrage, sinon Portainer se verrouille et il faut redémarrer le conteneur. Le montage du socket Docker donne à Portainer un pouvoir total sur l'hôte : en production, réservez-lui une instance dédiée avec HTTPS propre et des comptes nominatifs.

NOTES

Quiz de clôture — Jour 2

#	QUESTION	VOTRE RÉPONSE
1	Volume nommé ou bind mount pour les données PostgreSQL en production ?	
2	Et pour le code source en développement ?	
3	<code>docker compose down -v</code> : que fait le <code>-v</code> ?	
4	Pourquoi « localhost » ne marche-t-il jamais entre deux conteneurs ?	
5	Que garantit <code>depends_on: condition: service_healthy</code> ?	
6	Quelle commande valide un <code>compose.yaml</code> sans rien lancer ?	
7	À quoi sert <code>profiles: ["debug"]</code> sur un service ?	
8	Pourquoi <code>--scale api=3</code> a-t-il fonctionné sans conflit de ports ?	